

# Algorithms Time & Space Complexity

## Sorting Algorithms

| Algorithm             | Best Time     | Average Time  | Worst Time    | Space       | Stable | Notes                                |
|-----------------------|---------------|---------------|---------------|-------------|--------|--------------------------------------|
| <b>Bubble Sort</b>    | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$      | ✓      | Good for nearly sorted data          |
| <b>Selection Sort</b> | $O(n^2)$      | $O(n^2)$      | $O(n^2)$      | $O(1)$      | ✗      | Always $O(n^2)$ comparisons          |
| <b>Insertion Sort</b> | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$      | ✓      | Efficient for small/nearly sorted    |
| <b>Merge Sort</b>     | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(n)$      | ✓      | Divide and conquer                   |
| <b>Quick Sort</b>     | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$      | $O(\log n)$ | ✗      | Fast in practice, good pivot matters |
| <b>Heap Sort</b>      | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(1)$      | ✗      | In-place, not stable                 |
| <b>Counting Sort</b>  | $O(n + k)$    | $O(n + k)$    | $O(n + k)$    | $O(k)$      | ✓      | $k$ = range of input, for integers   |
| <b>Radix Sort</b>     | $O(d(n + k))$ | $O(d(n + k))$ | $O(d(n + k))$ | $O(n + k)$  | ✓      | $d$ = digits, $k$ = base             |
| <b>Bucket Sort</b>    | $O(n + k)$    | $O(n + k)$    | $O(n^2)$      | $O(n + k)$  | ✓      | Uniformly distributed input          |
| <b>Tim Sort</b>       | $O(n)$        | $O(n \log n)$ | $O(n \log n)$ | $O(n)$      | ✓      | Python/Java default                  |

## Searching Algorithms

| Algorithm                        | Time Complexity                       | Space       | Prerequisites                 | Notes                              |
|----------------------------------|---------------------------------------|-------------|-------------------------------|------------------------------------|
| <b>Linear Search</b>             | $O(n)$                                | $O(1)$      | None                          | Check each element                 |
| <b>Binary Search</b>             | $O(\log n)$                           | $O(1)$      | Sorted array                  | Divide and conquer                 |
| <b>Binary Search (Recursive)</b> | $O(\log n)$                           | $O(\log n)$ | Sorted array                  | Stack space for recursion          |
| <b>Jump Search</b>               | $O(\sqrt{n})$                         | $O(1)$      | Sorted array                  | Jump by $\sqrt{n}$ steps           |
| <b>Interpolation Search</b>      | $O(\log \log n)$ avg,<br>$O(n)$ worst | $O(1)$      | Sorted, uniformly distributed | Like binary but estimates position |

| Algorithm                 | Time Complexity | Space  | Prerequisites | Notes                          |
|---------------------------|-----------------|--------|---------------|--------------------------------|
| <b>Exponential Search</b> | $O(\log n)$     | $O(1)$ | Sorted array  | Finds range then binary search |
| <b>Ternary Search</b>     | $O(\log_3 n)$   | $O(1)$ | Sorted array  | Divide into 3 parts            |

## Graph Algorithms

| Algorithm                         | Time Complexity     | Space    | Notes                                         |
|-----------------------------------|---------------------|----------|-----------------------------------------------|
| <b>BFS (Breadth-First Search)</b> | $O(V + E)$          | $O(V)$   | Queue-based, shortest path unweighted         |
| <b>DFS (Depth-First Search)</b>   | $O(V + E)$          | $O(V)$   | Stack-based, detect cycles                    |
| <b>Dijkstra's Algorithm</b>       | $O((V + E) \log V)$ | $O(V)$   | Shortest path, non-negative weights, min heap |
| <b>Dijkstra (Array)</b>           | $O(V^2)$            | $O(V)$   | Shortest path, dense graphs                   |
| <b>Bellman-Ford</b>               | $O(VE)$             | $O(V)$   | Shortest path, handles negative weights       |
| <b>Floyd-Warshall</b>             | $O(V^3)$            | $O(V^2)$ | All pairs shortest path                       |
| <b>Prim's Algorithm</b>           | $O((V + E) \log V)$ | $O(V)$   | Minimum spanning tree, min heap               |
| <b>Prim's (Array)</b>             | $O(V^2)$            | $O(V)$   | MST, dense graphs                             |
| <b>Kruskal's Algorithm</b>        | $O(E \log E)$       | $O(V)$   | MST, union-find                               |
| <b>Topological Sort (DFS)</b>     | $O(V + E)$          | $O(V)$   | DAG only, stack-based                         |
| <b>Topological Sort (Kahn's)</b>  | $O(V + E)$          | $O(V)$   | DAG only, BFS-based                           |
| <b>Tarjan's SCC</b>               | $O(V + E)$          | $O(V)$   | Strongly connected components                 |
| <b>Kosaraju's SCC</b>             | $O(V + E)$          | $O(V)$   | SCC, two DFS passes                           |
| <i>A Search*</i>                  | $O(E)$              | $O(V)$   | Shortest path with heuristic                  |
| <b>Floyd's Cycle Detection</b>    | $O(n)$              | $O(1)$   | Detect cycle in linked list                   |

## Tree Algorithms

| Algorithm                         | Time Complexity               | Space  | Notes                                       |
|-----------------------------------|-------------------------------|--------|---------------------------------------------|
| <b>Tree Traversal (Inorder)</b>   | $O(n)$                        | $O(h)$ | $h$ = height, recursive stack               |
| <b>Tree Traversal (Preorder)</b>  | $O(n)$                        | $O(h)$ | Root $\rightarrow$ Left $\rightarrow$ Right |
| <b>Tree Traversal (Postorder)</b> | $O(n)$                        | $O(h)$ | Left $\rightarrow$ Right $\rightarrow$ Root |
| <b>Level Order Traversal</b>      | $O(n)$                        | $O(w)$ | $w$ = max width, BFS                        |
| <b>Binary Search (BST)</b>        | $O(\log n)$ avg, $O(n)$ worst | $O(1)$ | Balanced BST is $O(\log n)$                 |

| Algorithm                        | Time Complexity               | Space  | Notes                      |
|----------------------------------|-------------------------------|--------|----------------------------|
| <b>BST Insertion</b>             | $O(\log n)$ avg, $O(n)$ worst | $O(1)$ | Unbalanced can be $O(n)$   |
| <b>BST Deletion</b>              | $O(\log n)$ avg, $O(n)$ worst | $O(1)$ | Find successor/predecessor |
| <b>AVL Tree Operations</b>       | $O(\log n)$                   | $O(1)$ | Self-balancing BST         |
| <b>Red-Black Tree Operations</b> | $O(\log n)$                   | $O(1)$ | Self-balancing BST         |
| <b>Heap Insert</b>               | $O(\log n)$                   | $O(1)$ | Bubble up                  |
| <b>Heap Extract Min/Max</b>      | $O(\log n)$                   | $O(1)$ | Bubble down                |
| <b>Heapify</b>                   | $O(n)$                        | $O(1)$ | Build heap from array      |
| <b>Trie Insert/Search</b>        | $O(m)$                        | $O(m)$ | $m$ = key length           |
| <b>Segment Tree Build</b>        | $O(n)$                        | $O(n)$ | Range queries              |
| <b>Segment Tree Query</b>        | $O(\log n)$                   | $O(1)$ | Range sum/min/max          |
| <b>Fenwick Tree (BIT) Update</b> | $O(\log n)$                   | $O(n)$ | Prefix sums                |
| <b>Fenwick Tree Query</b>        | $O(\log n)$                   | $O(1)$ | Cumulative frequency       |

## Dynamic Programming Problems

| Problem                               | Time Complexity             | Space              | Pattern                         |
|---------------------------------------|-----------------------------|--------------------|---------------------------------|
| <b>Fibonacci (Recursive)</b>          | $O(2^n)$                    | $O(n)$             | Exponential without memoization |
| <b>Fibonacci (Memoization)</b>        | $O(n)$                      | $O(n)$             | Top-down DP                     |
| <b>Fibonacci (Tabulation)</b>         | $O(n)$                      | $O(n)$             | Bottom-up DP                    |
| <b>Fibonacci (Optimized)</b>          | $O(n)$                      | $O(1)$             | Only store last 2 values        |
| <b>0/1 Knapsack</b>                   | $O(nW)$                     | $O(nW)$            | $n$ items, $W$ capacity         |
| <b>Knapsack (Space Optimized)</b>     | $O(nW)$                     | $O(W)$             | 1D array                        |
| <b>Longest Common Subsequence</b>     | $O(mn)$                     | $O(mn)$            | $m, n$ = string lengths         |
| <b>LCS (Space Optimized)</b>          | $O(mn)$                     | $O(\min(m, n))$    | 1D array                        |
| <b>Longest Increasing Subsequence</b> | $O(n^2)$                    | $O(n)$             | DP solution                     |
| <b>LIS (Binary Search)</b>            | $O(n \log n)$               | $O(n)$             | Patience sorting                |
| <b>Edit Distance</b>                  | $O(mn)$                     | $O(mn)$            | Levenshtein distance            |
| <b>Coin Change</b>                    | $O(n \times \text{amount})$ | $O(\text{amount})$ | $n$ = coins                     |
| <b>Matrix Chain Multiplication</b>    | $O(n^3)$                    | $O(n^2)$           | Optimal parenthesization        |

| Problem                 | Time Complexity          | Space                    | Pattern                       |
|-------------------------|--------------------------|--------------------------|-------------------------------|
| Rod Cutting             | $O(n^2)$                 | $O(n)$                   | Max profit                    |
| Subset Sum              | $O(n \times \text{sum})$ | $O(n \times \text{sum})$ | NP-complete                   |
| Palindrome Partitioning | $O(n^2)$                 | $O(n^2)$                 | Min cuts                      |
| Word Break              | $O(n^2 \times m)$        | $O(n)$                   | $n$ = length, $m$ = dict size |

## String Algorithms

| Algorithm                   | Time Complexity               | Space      | Notes                                    |
|-----------------------------|-------------------------------|------------|------------------------------------------|
| Naive Pattern Matching      | $O(nm)$                       | $O(1)$     | $n$ = text, $m$ = pattern                |
| KMP (Knuth-Morris-Pratt)    | $O(n + m)$                    | $O(m)$     | Build LPS array                          |
| Rabin-Karp                  | $O(n + m)$ avg, $O(nm)$ worst | $O(1)$     | Rolling hash                             |
| Boyer-Moore                 | $O(n/m)$ best, $O(nm)$ worst  | $O(m)$     | Skip characters                          |
| Z Algorithm                 | $O(n + m)$                    | $O(n + m)$ | Pattern matching                         |
| Aho-Corasick                | $O(n + m + z)$                | $O(m)$     | Multiple pattern matching, $z$ = matches |
| Manacher's Algorithm        | $O(n)$                        | $O(n)$     | Longest palindromic substring            |
| Suffix Array                | $O(n \log n)$                 | $O(n)$     | Pattern matching, LCP                    |
| Suffix Tree                 | $O(n)$                        | $O(n)$     | Ukkonen's algorithm                      |
| Longest Palindrome (DP)     | $O(n^2)$                      | $O(n^2)$   | Dynamic programming                      |
| Longest Palindrome (Expand) | $O(n^2)$                      | $O(1)$     | Expand around center                     |

## Array/List Algorithms

| Algorithm             | Time Complexity | Space  | Notes              |
|-----------------------|-----------------|--------|--------------------|
| Two Pointers (Sorted) | $O(n)$          | $O(1)$ | Two sum, container |
| Sliding Window        | $O(n)$          | $O(k)$ | Max/min in window  |
| Kadane's Algorithm    | $O(n)$          | $O(1)$ | Max subarray sum   |
| Dutch National Flag   | $O(n)$          | $O(1)$ | 3-way partitioning |
| Boyer-Moore Voting    | $O(n)$          | $O(1)$ | Majority element   |

| Algorithm         | Time Complexity             | Space       | Notes                    |
|-------------------|-----------------------------|-------------|--------------------------|
| Quick Select      | $O(n)$ avg, $O(n^2)$ worst  | $O(1)$      | Kth largest element      |
| Median of Medians | $O(n)$                      | $O(\log n)$ | Guaranteed $O(n)$ select |
| Prefix Sum        | $O(n)$ build, $O(1)$ query  | $O(n)$      | Range sum queries        |
| Difference Array  | $O(n)$ build, $O(1)$ update | $O(n)$      | Range updates            |
| Monotonic Stack   | $O(n)$                      | $O(n)$      | Next greater element     |
| Monotonic Queue   | $O(n)$                      | $O(k)$      | Sliding window max       |

Mathematical Algorithms

| Algorithm                    | Time Complexity     | Space               | Notes                          |
|------------------------------|---------------------|---------------------|--------------------------------|
| GCD (Euclidean)              | $O(\log \min(a,b))$ | $O(1)$              | Greatest common divisor        |
| GCD (Recursive)              | $O(\log \min(a,b))$ | $O(\log \min(a,b))$ | Stack space                    |
| LCM                          | $O(\log \min(a,b))$ | $O(1)$              | Uses GCD                       |
| Prime Check (Trial Division) | $O(\sqrt{n})$       | $O(1)$              | Check divisibility             |
| Prime Check (Miller-Rabin)   | $O(k \log^3 n)$     | $O(1)$              | Probabilistic, k iterations    |
| Sieve of Eratosthenes        | $O(n \log \log n)$  | $O(n)$              | All primes up to n             |
| Segmented Sieve              | $O(n \log \log n)$  | $O(\sqrt{n})$       | Primes in range                |
| Prime Factorization          | $O(\sqrt{n})$       | $O(\log n)$         | Trial division                 |
| Modular Exponentiation       | $O(\log n)$         | $O(1)$              | $(a^b) \% m$                   |
| Fast Exponentiation          | $O(\log n)$         | $O(1)$              | $a^b$                          |
| Matrix Exponentiation        | $O(m^3 \log n)$     | $O(m^2)$            | $m \times m$ matrix to power n |
| Fibonacci (Matrix)           | $O(\log n)$         | $O(1)$              | Using matrix exponentiation    |

Backtracking Algorithms

| Algorithm      | Time Complexity       | Space    | Notes                       |
|----------------|-----------------------|----------|-----------------------------|
| N-Queens       | $O(n!)$               | $O(n^2)$ | Place n queens              |
| Sudoku Solver  | $O(9^{(n \times n)})$ | $O(n^2)$ | n = grid size (typically 9) |
| Permutations   | $O(n \times n!)$      | $O(n)$   | Generate all permutations   |
| Combinations   | $O(2^n)$              | $O(n)$   | Generate all subsets        |
| Subset Sum     | $O(2^n)$              | $O(n)$   | Find subset with sum        |
| Graph Coloring | $O(m^n)$              | $O(n)$   | m colors, n vertices        |

| Algorithm               | Time Complexity            | Space  | Notes                                |
|-------------------------|----------------------------|--------|--------------------------------------|
| <b>Hamiltonian Path</b> | $O(n!)$                    | $O(n)$ | Visit all vertices once              |
| <b>Word Search</b>      | $O(m \times n \times 4^L)$ | $O(L)$ | $m \times n$ grid, $L$ = word length |

## Greedy Algorithms

| Algorithm                  | Time Complexity | Space  | Notes                     |
|----------------------------|-----------------|--------|---------------------------|
| <b>Activity Selection</b>  | $O(n \log n)$   | $O(1)$ | Sort by finish time       |
| <b>Fractional Knapsack</b> | $O(n \log n)$   | $O(1)$ | Sort by value/weight      |
| <b>Huffman Coding</b>      | $O(n \log n)$   | $O(n)$ | Min heap for frequencies  |
| <b>Job Sequencing</b>      | $O(n^2 \log n)$ | $O(n)$ | Maximize profit           |
| <b>Interval Scheduling</b> | $O(n \log n)$   | $O(1)$ | Non-overlapping intervals |

## Bit Manipulation

| Operation                | Time Complexity | Space  | Notes                       |
|--------------------------|-----------------|--------|-----------------------------|
| <b>Count Set Bits</b>    | $O(\log n)$     | $O(1)$ | Brian Kernighan's algorithm |
| <b>Check Power of 2</b>  | $O(1)$          | $O(1)$ | $n \& (n-1) == 0$           |
| <b>Get/Set/Clear Bit</b> | $O(1)$          | $O(1)$ | Bit operations              |
| <b>XOR All Elements</b>  | $O(n)$          | $O(1)$ | Find unique element         |
| <b>Generate Subsets</b>  | $O(2^n)$        | $O(1)$ | Use bitmask                 |

## Advanced Data Structures

| Data Structure            | Insert          | Delete          | Search          | Space                                  | Notes                             |
|---------------------------|-----------------|-----------------|-----------------|----------------------------------------|-----------------------------------|
| <b>Hash Table</b>         | $O(1)$ avg      | $O(1)$ avg      | $O(1)$ avg      | $O(n)$                                 | Worst case $O(n)$ with collisions |
| <b>Binary Search Tree</b> | $O(\log n)$ avg | $O(\log n)$ avg | $O(\log n)$ avg | $O(n)$                                 | Worst $O(n)$ if unbalanced        |
| <b>AVL Tree</b>           | $O(\log n)$     | $O(\log n)$     | $O(\log n)$     | $O(n)$                                 | Self-balancing                    |
| <b>Red-Black Tree</b>     | $O(\log n)$     | $O(\log n)$     | $O(\log n)$     | $O(n)$                                 | Self-balancing                    |
| <b>B-Tree</b>             | $O(\log n)$     | $O(\log n)$     | $O(\log n)$     | $O(n)$                                 | Disk-based, multiple children     |
| <b>Trie</b>               | $O(m)$          | $O(m)$          | $O(m)$          | $O(\text{ALPHABET} \times N \times M)$ | $m$ = key length                  |

| Data Structure            | Insert             | Delete             | Search             | Space  | Notes                                  |
|---------------------------|--------------------|--------------------|--------------------|--------|----------------------------------------|
| Segment Tree              | $O(n)$<br>build    | -                  | $O(\log n)$        | $O(n)$ | Range queries                          |
| Fenwick Tree (BIT)        | $O(\log n)$        | -                  | $O(\log n)$        | $O(n)$ | Prefix sums                            |
| Disjoint Set (Union-Find) | $O(\alpha(n))$     | -                  | $O(\alpha(n))$     | $O(n)$ | $\alpha$ = inverse Ackermann           |
| Min/Max Heap              | $O(\log n)$        | $O(\log n)$        | $O(1)$<br>peek     | $O(n)$ | Priority queue                         |
| Skip List                 | $O(\log n)$<br>avg | $O(\log n)$<br>avg | $O(\log n)$<br>avg | $O(n)$ | Probabilistic balancing                |
| Bloom Filter              | $O(k)$             | -                  | $O(k)$             | $O(m)$ | $k$ = hash functions,<br>probabilistic |

Divide & Conquer

| Algorithm                | Time Complexity   | Space         | Recurrence                |
|--------------------------|-------------------|---------------|---------------------------|
| Binary Search            | $O(\log n)$       | $O(1)$        | $T(n) = T(n/2) + O(1)$    |
| Merge Sort               | $O(n \log n)$     | $O(n)$        | $T(n) = 2T(n/2) + O(n)$   |
| Quick Sort               | $O(n \log n)$ avg | $O(\log n)$   | $T(n) = 2T(n/2) + O(n)$   |
| Closest Pair of Points   | $O(n \log n)$     | $O(n)$        | $T(n) = 2T(n/2) + O(n)$   |
| Strassen's Matrix Mult   | $O(n^{2.807})$    | $O(n^2)$      | $T(n) = 7T(n/2) + O(n^2)$ |
| Karatsuba Multiplication | $O(n^{1.585})$    | $O(n \log n)$ | $T(n) = 3T(n/2) + O(n)$   |

Two Pointers Patterns

| Pattern                    | Time Complexity | Space  | Use Case                           |
|----------------------------|-----------------|--------|------------------------------------|
| Opposite Direction         | $O(n)$          | $O(1)$ | Two sum sorted, palindrome         |
| Same Direction (Fast/Slow) | $O(n)$          | $O(1)$ | Remove duplicates, cycle detection |
| Sliding Window (Fixed)     | $O(n)$          | $O(1)$ | Max sum $k$ elements               |
| Sliding Window (Variable)  | $O(n)$          | $O(k)$ | Longest substring                  |

Master Theorem Quick Reference

For recurrence:  $T(n) = aT(n/b) + f(n)$

| Case | Condition                      | Result                         |
|------|--------------------------------|--------------------------------|
| 1    | $f(n) = O(n^c), c < \log_b(a)$ | $T(n) = \Theta(n^{\log_b(a)})$ |

| Case | Condition                                       | Result                              |
|------|-------------------------------------------------|-------------------------------------|
| 2    | $f(n) = \Theta(n^c \log^k n)$ , $c = \log_b(a)$ | $T(n) = \Theta(n^c \log^{(k+1)} n)$ |
| 3    | $f(n) = \Omega(n^c)$ , $c > \log_b(a)$          | $T(n) = \Theta(f(n))$               |

## Common Complexity Classes

| Class         | Name         | Example                               |
|---------------|--------------|---------------------------------------|
| $O(1)$        | Constant     | Hash table lookup                     |
| $O(\log n)$   | Logarithmic  | Binary search                         |
| $O(n)$        | Linear       | Array traversal                       |
| $O(n \log n)$ | Linearithmic | Merge sort, quick sort                |
| $O(n^2)$      | Quadratic    | Bubble sort, nested loops             |
| $O(n^3)$      | Cubic        | Floyd-Warshall, matrix multiplication |
| $O(2^n)$      | Exponential  | Recursive fibonacci, subsets          |
| $O(n!)$       | Factorial    | Permutations, TSP brute force         |
| $O(n^n)$      | N to the N   | Generating all passwords              |